TP4-ADL

Mathura SANTHALINGAM (12212867)

Partie 2. Charger le jeu de données

Question 1 :

```
wine <- read.csv("wine.data", header = FALSE)

head(wine)
```

	V1 <int>	V2 <dbl>	V3 <dbl>	V4 <dbl>	V5 <dbl>	V6 <int>	V7 <dbl>	V8 <dbl>	V9 <dbl>
1	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28
2	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26
3	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30
4	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24
5	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39
6	1	14.20	1.76	2.45	15.2	112	3.27	3.39	0.34

6 rows | 1-10 of 15 columns

Question 2 :

```
colnames(wine) <- c("Customer_Segment","Alcohol","Malic_Acid","Ash","Ash_Alkanity",  "Magnesium","Total_Phenols",
"Flavanoids","Nonflavanoid_Phenols", "Proanthocyanins",  "Color_Intensity", "Hue",  "OD280","Proline")
head(wine)
```

	Customer_Segment <int>	Alcohol <dbl>	Malic_Acid <dbl>	Ash <dbl>	Ash_Alkanity <dbl>	Magnesium <int>	Total_Phenols <dbl>
1	1	14.23	1.71	2.43	15.6	127	2.80
2	1	13.20	1.78	2.14	11.2	100	2.65
3	1	13.16	2.36	2.67	18.6	101	2.80
4	1	14.37	1.95	2.50	16.8	113	3.85
5	1	13.24	2.59	2.87	21.0	118	2.80
6	1	14.20	1.76	2.45	15.2	112	3.27

6 rows | 1-8 of 15 columns

Question 3 :

```
labels <- wine$Customer_Segment
wine$Customer_Segment <- NULL
wine$Customer_Segment <- labels

head(wine)
```

	Alcohol <dbl>	Malic_Acid <dbl>	Ash <dbl>	Ash_Alkanity <dbl>	Magnesium <int>	Total_Phenols <dbl>	Flavanoids <dbl>
1	14.23	1.71	2.43	15.6	127	2.80	3.06
2	13.20	1.78	2.14	11.2	100	2.65	2.76
3	13.16	2.36	2.67	18.6	101	2.80	3.24
4	14.37	1.95	2.50	16.8	113	3.85	3.49
5	13.24	2.59	2.87	21.0	118	2.80	2.69
6	14.20	1.76	2.45	15.2	112	3.27	3.39

6 rows | 1-8 of 15 columns

Question 4 :

```
compteur <- 0
for (i in 1:nrow(wine)) {
  ligne <- wine[i, ]
  val_manquante <- FALSE
  for (j in 1:ncol(wine)) {
    if (is.na(ligne[j])) {
      val_manquante <- TRUE
    }
  }
  if (val_manquante) {
    compteur <- compteur + 1
  }
}
cat("Nombre de lignes avec des valeurs manquantes :", compteur, "\n")
```

```
## Nombre de lignes avec des valeurs manquantes : 0
```

Suppression des lignes contenant des valeurs manquantes :

```
wine_propre <- data.frame()
for (i in 1:nrow(wine)) {
  ligne <- wine[i, ]
  na_dans_ligne <- FALSE
  for (j in 1:ncol(wine)) {
    if (is.na(ligne[j])) {
      na_dans_ligne <- TRUE
    }
  }
  if (!na_dans_ligne) {
    wine_propre <- rbind(wine_propre, ligne)
  }
}
wine <- wine_propre
cat("Nombre de lignes après suppression des valeurs manquantes :", nrow(wine), "\n")
```

```
## Nombre de lignes après suppression des valeurs manquantes : 178
```

Question 5 :

```
summary(wine)
```

```
##     Alcohol        Malic_Acid         Ash         Ash_Alkanity
##  Min.   :11.03   Min.   :0.740   Min.   :1.360   Min.   :10.60
##  1st Qu.:12.36   1st Qu.:1.603   1st Qu.:2.210   1st Qu.:17.20
##  Median :13.05   Median :1.865   Median :2.360   Median :19.50
##  Mean   :13.00   Mean   :2.336   Mean   :2.367   Mean   :19.49
##  3rd Qu.:13.68   3rd Qu.:3.083   3rd Qu.:2.558   3rd Qu.:21.50
##  Max.   :14.83   Max.   :5.800   Max.   :3.230   Max.   :30.00
##    Magnesium      Total_Phenols     Flavanoids    Nonflavanoid_Phenols
##  Min.   : 70.00   Min.   :0.980   Min.   :0.340   Min.   :0.1300
##  1st Qu.: 88.00   1st Qu.:1.742   1st Qu.:1.205   1st Qu.:0.2700
##  Median : 98.00   Median :2.355   Median :2.135   Median :0.3400
##  Mean   : 99.74   Mean   :2.295   Mean   :2.029   Mean   :0.3619
##  3rd Qu.:107.00   3rd Qu.:2.800   3rd Qu.:2.875   3rd Qu.:0.4375
##  Max.   :162.00   Max.   :3.880   Max.   :5.080   Max.   :0.6600
##  Proanthocyanins Color_Intensity       Hue            OD280
##  Min.   :0.410   Min.   : 1.280   Min.   :0.4800   Min.   :1.270
##  1st Qu.:1.250   1st Qu.: 3.220   1st Qu.:0.7825   1st Qu.:1.938
##  Median :1.555   Median : 4.690   Median :0.9650   Median :2.780
##  Mean   :1.591   Mean   : 5.058   Mean   :0.9574   Mean   :2.612
##  3rd Qu.:1.950   3rd Qu.: 6.200   3rd Qu.:1.1200   3rd Qu.:3.170
##  Max.   :3.580   Max.   :13.000   Max.   :1.7100   Max.   :4.000
##     Proline       Customer_Segment
##  Min.   : 278.0   Min.   :1.000
##  1st Qu.: 500.5   1st Qu.:1.000
##  Median : 673.5   Median :2.000
##  Mean   : 746.9   Mean   :1.938
##  3rd Qu.: 985.0   3rd Qu.:3.000
##  Max.   :1680.0   Max.   :3.000
```

Question 6 :

```
#install.packages("caTools")
library(caTools)
```

```
set.seed(123)
div <- sample.split(wine$Customer_Segment, SplitRatio = 0.8)
entrainement <- subset(wine, div == TRUE)
test <- subset(wine, div == FALSE)
nrow(entrainement)
```

```
## [1] 142
```

```
nrow(test)
```

```
## [1] 36
```

Question 7 :

```
# Normaliser seulement les colonnes numériques du train set (sauf Customer_Segment)
entrainement_norm <- scale(entrainement[, -ncol(entrainement)])
entrainement_norm <- data.frame(entrainement_norm, Customer_Segment = entrainement$Customer_Segment)


# Normaliser seulement les colonnes numériques du test set (sauf Customer_Segment)
test_norm <- scale(test[, -ncol(test)])
test_norm <- data.frame(test_norm, Customer_Segment = test$Customer_Segment)
```

Chaque valeur d'une variable est transformée en soustrayant la moyenne de cette variable puis en divisant par son écart-type. On a ainsi que la moyenne de chaque variable devient 0 et l'écart-type devient 1 Cette transformation permet de rendre toutes les variables comparables.

– Partie 3. Construire un model SVM pour la tâche de classification des vins selon leur producteur

Question 1 :

```
#install.packages("e1071")
library(e1071)
```

```
entrainement_norm$Customer_Segment <- as.factor(entrainement_norm$Customer_Segment)
test_norm$Customer_Segment <- as.factor(test_norm$Customer_Segment)
svm_model <- svm(Customer_Segment ~ ., data = entrainement_norm, kernel = "linear")
```

Question 2 :

```
predictions <- predict(svm_model, newdata = test_norm)
predictions
```

```
##   4   5   8  11  16  20  21  24  31  32  50  59  65  67  68  69  87  88  89 104
##   1   1   1   1   1   1   1   1   1   1   1   1   2   2   2   3   2   2   2   2
## 106 107 111 114 118 126 132 134 137 138 139 145 151 167 173 174
##   2   2   2   2   2   2   3   3   3   3   3   3   3   3   3   3
## Levels: 1 2 3
```

Question 3 : Une matrice de confusion est un tableau qui permet de comparer les prédictions d'un modèle de classification avec les vraies valeurs : - Les lignes représentent les vraies classes des observations. - Les colonnes représentent les classes prédites par le modèle. Chaque case indique combien d'observations d'une classe réelle ont été prédites dans chaque classe. Elle permet de visualiser les erreurs de classification et de mesurer la performance du modèle.

Question 4 : - Accuracy : proportion d'observations correctement classées sur l'ensemble des observations Formule : Nombre de prédictions correctes / Nombre total de prédictions

- Precision : Pour une classe donnée c'est la proportion de prédictions correctes parmi toutes les prédictions de cette classe Formule: vrais positifs / (vrais positifs + faux positifs)
- Recall : Pour une classe donnée c'est la proportion d'observations correctement prédites parmi toutes les observations réelles de cette classe. Formule : vrais positifs / (vrais positifs + faux négatifs)

F1-score : Moyenne harmonique de la précision et du recall. Formule : 2 x (précision x recall/ (précision + recall))

Question 5 :

```
confusion <- table(Actual = test_norm$Customer_Segment,
                   Predicted = predictions)
confusion
```

```
##       Predicted
## Actual  1  2  3
##      1 12  0  0
##      2  0 13  1
##      3  0  0 10
```

```
accuracy <- sum(diag(confusion)) / sum(confusion)
accuracy
```

```
## [1] 0.9722222
```

```
classes <- rownames(confusion)

metrics <- data.frame(Classe = classes, Precision = numeric(length(classes)), Recall = numeric(length(classes)),
F1 = numeric(length(classes)), Support = numeric(length(classes)))

for (i in seq_along(classes)) {
  cls <- classes[i]
  TP <- confusion[cls, cls]
  FP <- sum(confusion[, cls]) - TP
  FN <- sum(confusion[cls, ]) - TP

  precision_i <- if ((TP + FP) == 0) 0 else TP / (TP + FP)
  recall_i    <- if ((TP + FN) == 0) 0 else TP / (TP + FN)
  f1_i        <- if ((precision_i + recall_i) == 0) 0 else
    (2 * precision_i * recall_i) / (precision_i + recall_i)

  metrics$Precision[i] <- round(precision_i, 4)
  metrics$Recall[i]    <- round(recall_i, 4)
  metrics$F1[i]        <- round(f1_i, 4)
  metrics$Support[i]   <- sum(confusion[cls, ])
}

metrics
```

Classe <chr>	Precision <dbl>	Recall <dbl>	F1 <dbl>	Support <dbl>
1	1.0000	1.0000	1.0000	12
2	1.0000	0.9286	0.9630	14
3	0.9091	1.0000	0.9524	10

3 rows

Partie 4. Réduction de dimensionalité à l'aide de l'ACP Question 1 :

```
#install.packages("caret")
#install.packages("e1071")
library(caret)
```

```
## Loading required package: ggplot2
```

```
##
## Attaching package: 'ggplot2'
```

```
## The following object is masked from 'package:e1071':
##
##     element
```

```
## Loading required package: lattice
```

```
library(e1071)
```

Question 2 :

```
modele_avec_pca <- prcomp(entrainement_norm[, -ncol(entrainement_norm)], center = TRUE, scale. = TRUE)
summary(modele_avec_pca)
```

```
## Importance of components:
##                           PC1    PC2    PC3     PC4     PC5     PC6     PC7
## Standard deviation     2.1617 1.5487 1.2397 0.97904 0.91350 0.82246 0.75088
## Proportion of Variance 0.3595 0.1845 0.1182 0.07373 0.06419 0.05203 0.04337
## Cumulative Proportion  0.3595 0.5440 0.6622 0.73594 0.80013 0.85216 0.89553
##                            PC8     PC9    PC10   PC11    PC12   PC13
## Standard deviation     0.60288 0.54635 0.47809 0.4646 0.37915 0.3286
## Proportion of Variance 0.02796 0.02296 0.01758 0.0166 0.01106 0.0083
## Cumulative Proportion  0.92349 0.94645 0.96404 0.9806 0.99170 1.0000
```

```
train_set_PCA <- data.frame(modele_avec_pca$x[, 1:2], Customer_Segment = entrainement_norm$Customer_Segment)

test_set_PCA_valeur <- predict(modele_avec_pca, newdata = test_norm[, -ncol(test_norm)])
test_set_PCA <- data.frame(test_set_PCA_valeur[, 1:2], Customer_Segment = test_norm$Customer_Segment)
```

Question 3 :

```
head(train_set_PCA)
```

	PC1	PC2	Customer_Segment
	<dbl>	<dbl>	<fct>
1	-3.249569	-1.5661160	1
2	-2.165889	0.3186768	1
3	-2.501192	-1.2353892	1
6	-2.941040	-2.2999654	1
7	-2.393131	-1.3228050	1
9	-2.418465	-1.0367916	1

6 rows

```
head(test_set_PCA)
```

	PC1 <dbl>	PC2 <dbl>	Customer_Segment <fct>
4	-3.481905	-2.7632899	1
5	-1.036443	-0.9838128	1
8	-1.986189	-1.5520613	1
11	-3.329896	-1.2446480	1
16	-2.236477	-1.6347229	1
20	-2.112082	-1.0008301	1

6 rows

Question 4 :

```
svm_pca <- svm(Customer_Segment ~ ., data = train_set_PCA, kernel = "linear")
```

Question 5 :

```
predictions_pca <- predict(svm_pca, newdata = test_set_PCA)
head(predictions_pca)
```

```
##  4  5  8 11 16 20
##  1  1  1  1  1  1
## Levels: 1 2 3
```

Question 6 :

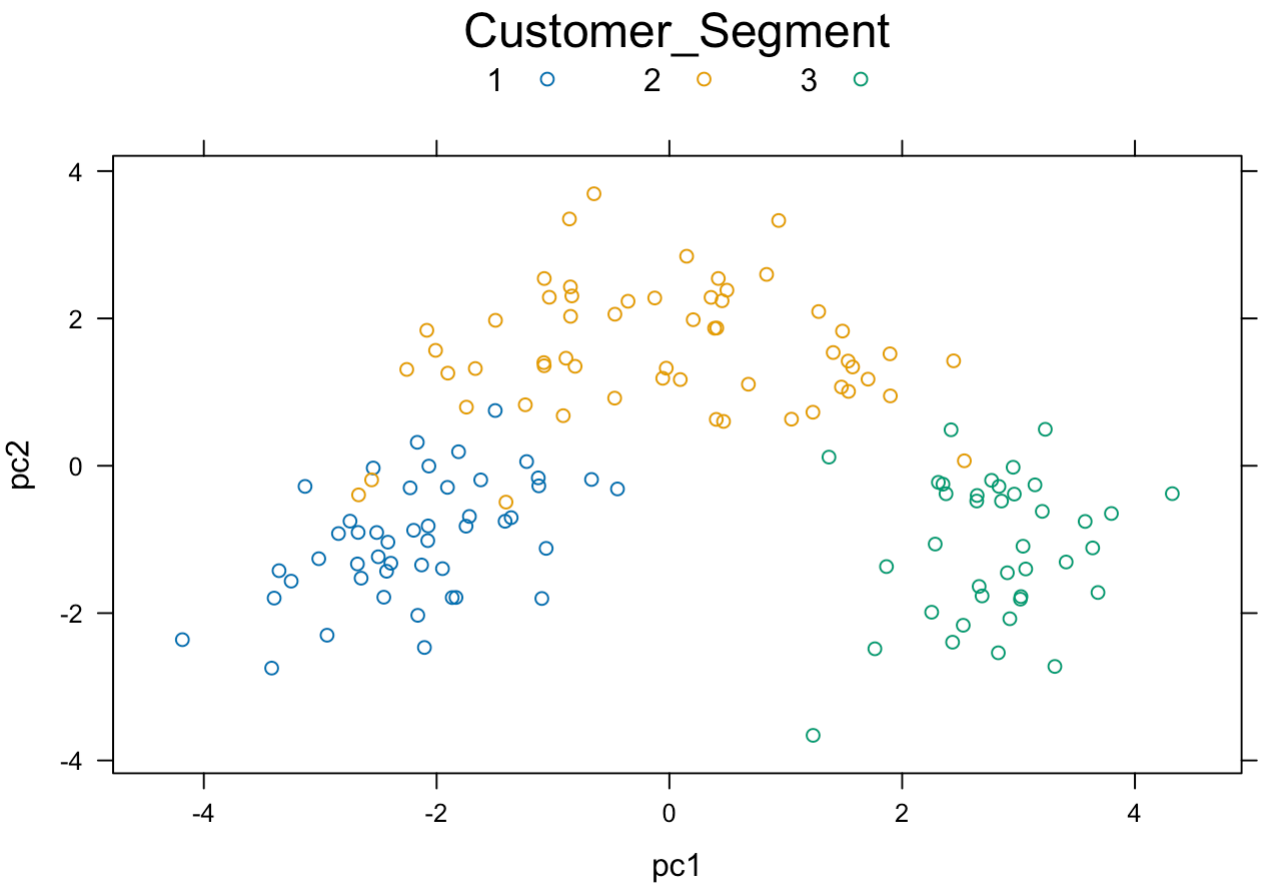
```
matrice_confusion_pca <- confusionMatrix(predictions_pca, test_set_PCA$Customer_Segment)
matrice_confusion_pca
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1  2  3
##          1 12  0  0
##          2  0 14  0
##          3  0  0 10
##
## Overall Statistics
##
##                Accuracy : 1
##                  95% CI : (0.9026, 1)
##     No Information Rate : 0.3889
##     P-Value [Acc > NIR] : 1.713e-15
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            1.0000   1.0000   1.0000
## Specificity            1.0000   1.0000   1.0000
## Pos Pred Value         1.0000   1.0000   1.0000
## Neg Pred Value         1.0000   1.0000   1.0000
## Prevalence             0.3333   0.3889   0.2778
## Detection Rate         0.3333   0.3889   0.2778
## Detection Prevalence   0.3333   0.3889   0.2778
## Balanced Accuracy      1.0000   1.0000   1.0000
```

Question 7 :

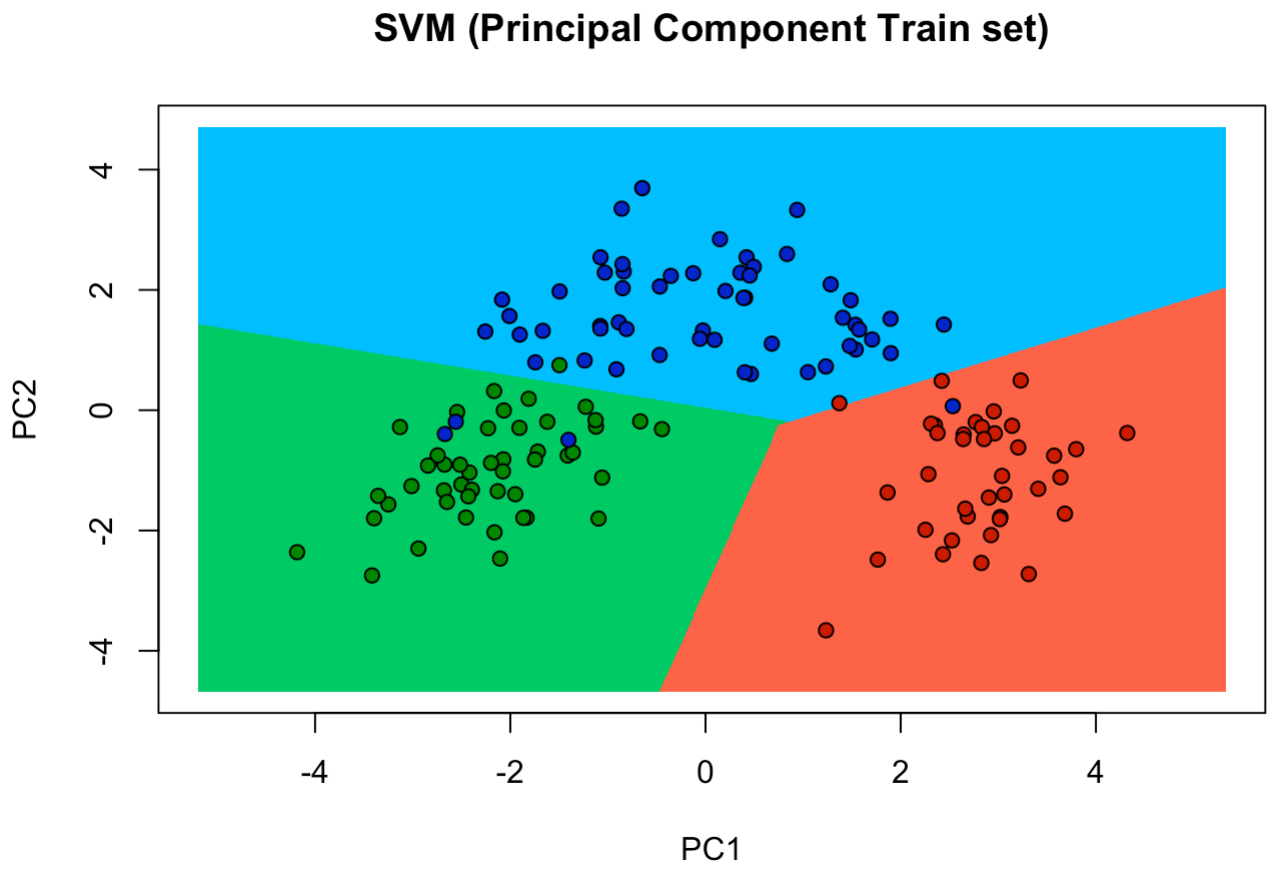
```
library(lattice)
```

```
xyplot(PC2 ~ PC1, data = train_set_PCA, groups = Customer_Segment, auto.key = list(title = "Customer_Segment", co
lumns = 3), xlab = "pc1", ylab = "pc2",)
```



Question 8 :

```
set = train_set_PCA
X1 = seq(min(set$PC1) - 1, max(set$PC1) + 1, by = 0.01)
X2 = seq(min(set$PC2) - 1, max(set$PC2) + 1, by = 0.01)
grid_set = expand.grid(PC1 = X1, PC2 = X2)
y_grid = predict(svm_pca, newdata = grid_set)
plot(set$PC1, set$PC2,
     main = 'SVM (Principal Component Train set)',
     xlab = 'PC1', ylab = 'PC2',
     xlim = range(X1), ylim = range(X2))
points(grid_set, pch = '.', col = ifelse(y_grid == 2, 'deepskyblue', ifelse(y_grid == 1, 'springgreen3', 'tomat
o')))
points(set, pch = 21, bg = ifelse(set[, 3] == 2, 'blue3', ifelse(set[, 3] == 1, 'green4', 'red3')))
```



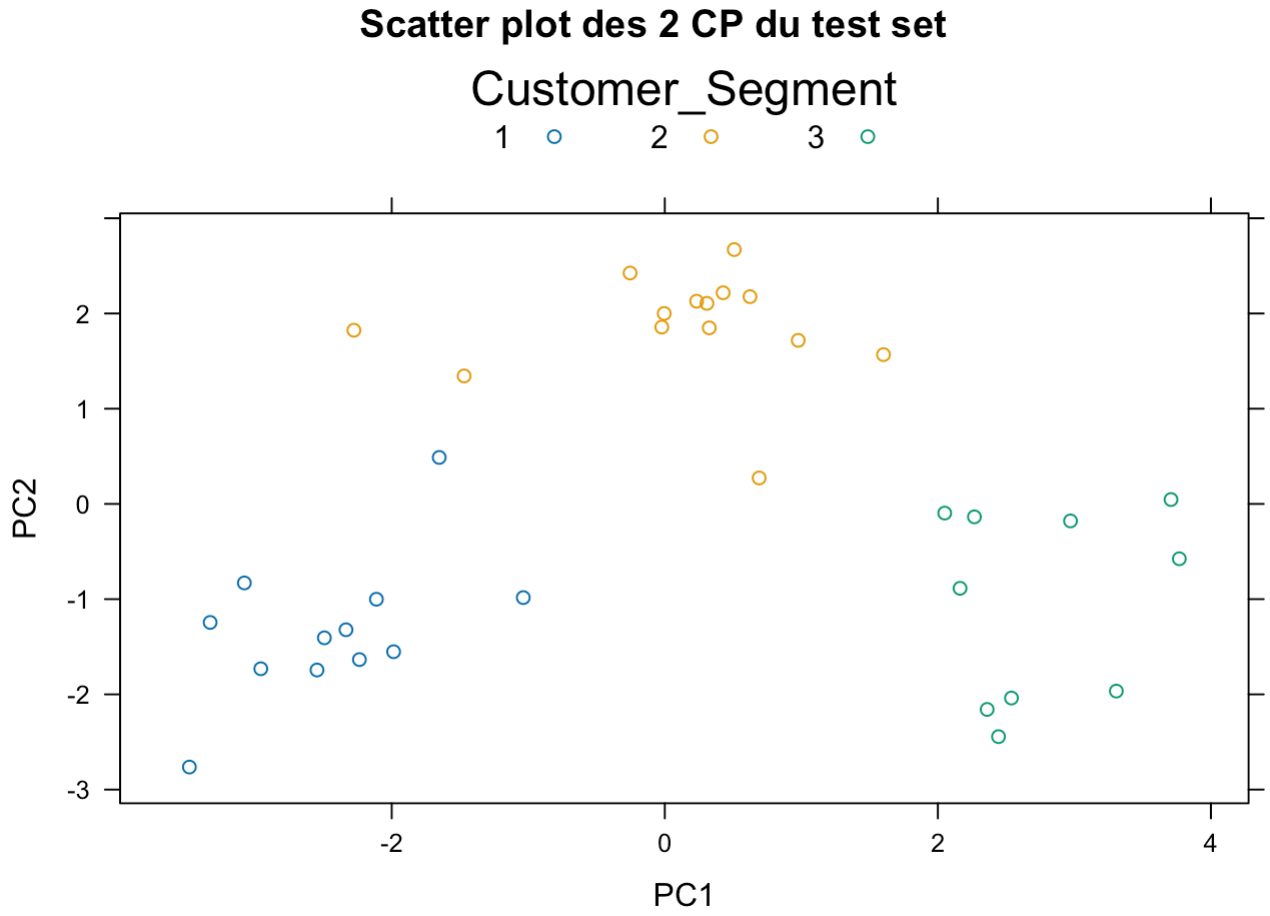
Il permet de visualiser les frontières de décision du modèle SVM dans l'espace réduit des deux composantes principales. Concrètement, une grille de points couvrant l'ensemble du plan est générée afin que le modèle en prédise la classe, ce qui permet de colorer le fond selon les zones de décision.

Les données d'entraînement réelles sont ensuite superposées sous forme de points colorés par classe. Cette représentation graphique offre une double lecture : elle met en évidence la manière dont le classifieur sépare les différentes catégories et permet d'évaluer visuellement la qualité de la classification.

Question 9 :

```
library(lattice)

xyplot(PC2 ~ PC1, data = test_set_PCA,
       groups = Customer_Segment,
       auto.key = list(title = "Customer_Segment", columns = 3),
       xlab = "PC1",
       ylab = "PC2",
       main = "Scatter plot des 2 CP du test set")
```

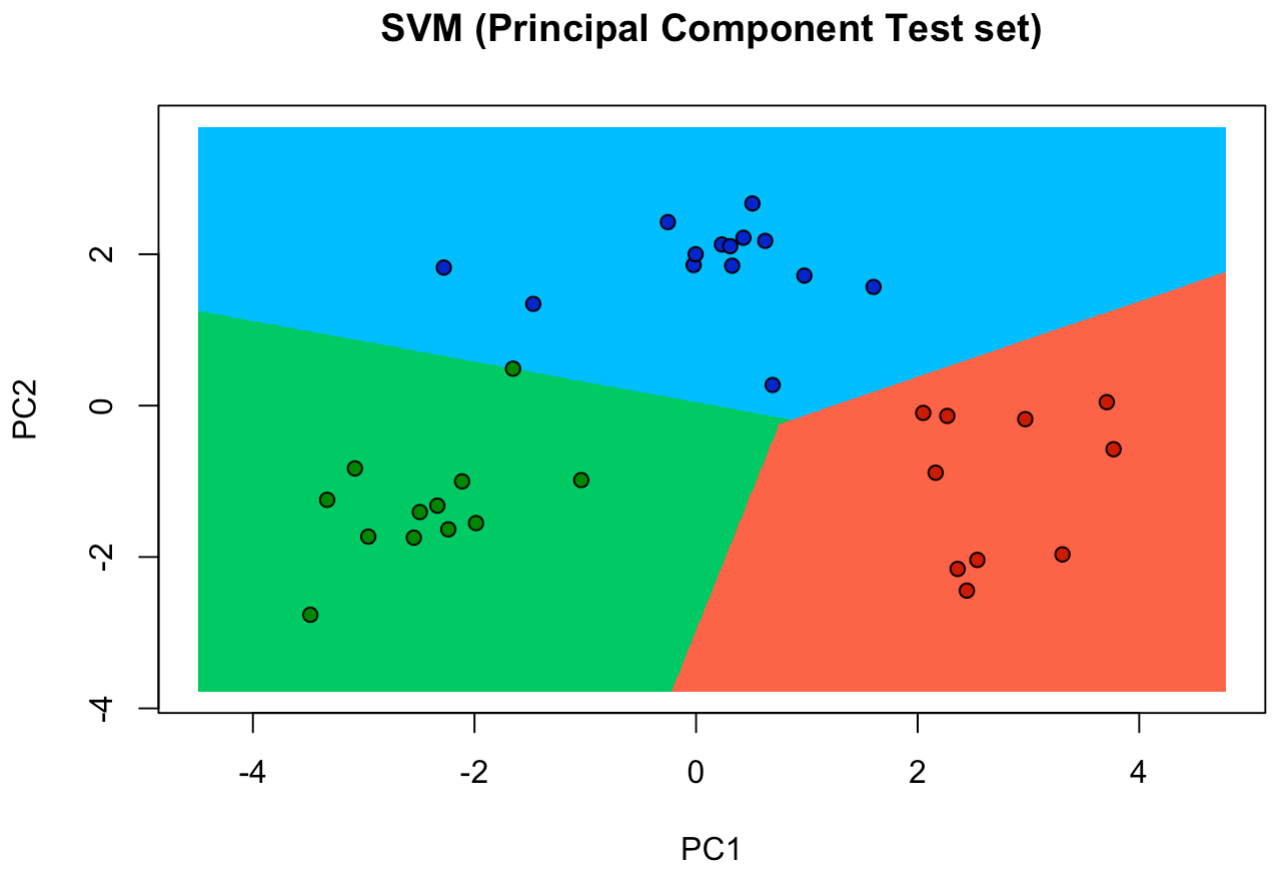


```
set = test_set_PCA
X1 = seq(min(set$PC1) - 1, max(set$PC1) + 1, by = 0.01)
X2 = seq(min(set$PC2) - 1, max(set$PC2) + 1, by = 0.01)
grid_set = expand.grid(PC1 = X1, PC2 = X2)
y_grid = predict(svm_pca, newdata = grid_set)

plot(set$PC1, set$PC2,
     main = 'SVM (Principal Component Test set)',
     xlab = 'PC1', ylab = 'PC2',
     xlim = range(X1), ylim = range(X2))

points(grid_set, pch = '.', col = ifelse(y_grid == 1, 'springgreen3', ifelse(y_grid == 2, 'deepskyblue', 'tomat
o')))

points(set$PC1, set$PC2, pch = 21, bg = ifelse(set$Customer_Segment == 1, 'green4', ifelse(set$Customer_Segment =
= 2, 'blue3', 'red3')))
```



On observe un regroupement des observations de chaque classe dans leurs zones de décision, ce qui montre la grande performance du modèle SVM. Ce constat est confirmé par la matrice de confusion et les métriques associées, qui affichent des scores égaux à 1 (parfaits). Les deux CP suffisent donc à séparer de manière idéale les trois classes de ce jeu de données.

Question 10 : C'est impossible de représenter graphiquement le modèle à partir des données d'origine, car celles-ci comportent treize variables. Étant donné qu'un plan offre au maximum deux dimensions, la visualisation simultanée de l'ensemble des caractéristiques est impossible. La réduction de dimension par ACP résout cette limite résultant sur deux composantes principales, ce qui permet d'afficher directement les frontières de décision dans un espace en 2D.

Partie 5. Réduction de dimensionalité à l'aide de l'ALD

Question 1 :

```
#install.packages("MASS")
library(MASS)
```

```
modele_avec_lda <- lda(Customer_Segment ~ ., data = entrainement_norm)
```

Question 2 :

```
entrainement_lda <- predict(modele_avec_lda, newdata = entrainement_norm)
classes_entrainement <- entrainement_lda$class

test_lda <- predict(modele_avec_lda, newdata = test_norm)
classes_test <- test_lda$class
```

Question 3 :

```
head(classes_entrainement)
```

```
## [1] 1 1 1 1 1 1
## Levels: 1 2 3
```

```
head(classes_test)
```

```
## [1] 1 1 1 1 1 1
## Levels: 1 2 3
```

Question 4 :

```
entrainement_lda2 <- data.frame(LD1 = entrainement_lda$x[,1],
                           LD2 = entrainement_lda$x[,2],
                           Customer_Segment = entrainement_norm$Customer_Segment)
test_lda2 <- data.frame(LD1 = test_lda$x[,1],
                        LD2 = test_lda$x[,2],
                        Customer_Segment = test_norm$Customer_Segment)

head(entrainement_lda2)
```

	LD1 <dbl>	LD2 <dbl>	Customer_Segment <fct>
1	-4.656187	2.081444	1
2	-4.336729	1.267238	1
3	-3.292202	1.167575	1
6	-4.515987	3.265418	1
7	-4.627289	3.369602	1
9	-3.936469	1.967177	1

6 rows

```
head(test_lda2)
```

	LD1 <dbl>	LD2 <dbl>	Customer_Segment <fct>
4	-4.092791	3.879201	1
5	-1.252520	0.321187	1
8	-3.984139	3.009815	1
11	-4.619335	1.623783	1
16	-3.097161	2.723515	1

	LD1	LD2	Customer_Segment
	<dbl>	<dbl>	<fct>
20	-3.410850	1.881416	1

6 rows

Question 5 :

```
library(e1071)

entrainement_lda2$Customer_Segment <- as.factor(entrainement_lda2$Customer_Segment)
test_lda2$Customer_Segment <- as.factor(test_lda2$Customer_Segment)

svm_lda <- svm(Customer_Segment ~ ., data = entrainement_lda2, kernel = "linear")

predictions_lda <- predict(svm_lda, newdata = test_lda2)

head(predictions_lda)
```

```
##  4  5  8 11 16 20
##  1  1  1  1  1  1
## Levels: 1 2 3
```

Question 6 :

```
library(caret)

matrice_confusion_lda <- confusionMatrix(predictions_lda, test_lda2$Customer_Segment)
matrice_confusion_lda
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction  1  2  3
##          1 12  1  0
##          2  0 13  0
##          3  0  0 10
##
## Overall Statistics
##
##                Accuracy : 0.9722
##                  95% CI : (0.8547, 0.9993)
##     No Information Rate : 0.3889
##     P-Value [Acc > NIR] : 9.861e-14
##
##                   Kappa : 0.958
##
##  Mcnemar's Test P-Value : NA
##
## Statistics by Class:
##
##                      Class: 1 Class: 2 Class: 3
## Sensitivity            1.0000   0.9286   1.0000
## Specificity            0.9583   1.0000   1.0000
## Pos Pred Value         0.9231   1.0000   1.0000
## Neg Pred Value         1.0000   0.9565   1.0000
## Prevalence             0.3333   0.3889   0.2778
## Detection Rate         0.3333   0.3611   0.2778
## Detection Prevalence   0.3611   0.3611   0.2778
## Balanced Accuracy      0.9792   0.9643   1.0000
```

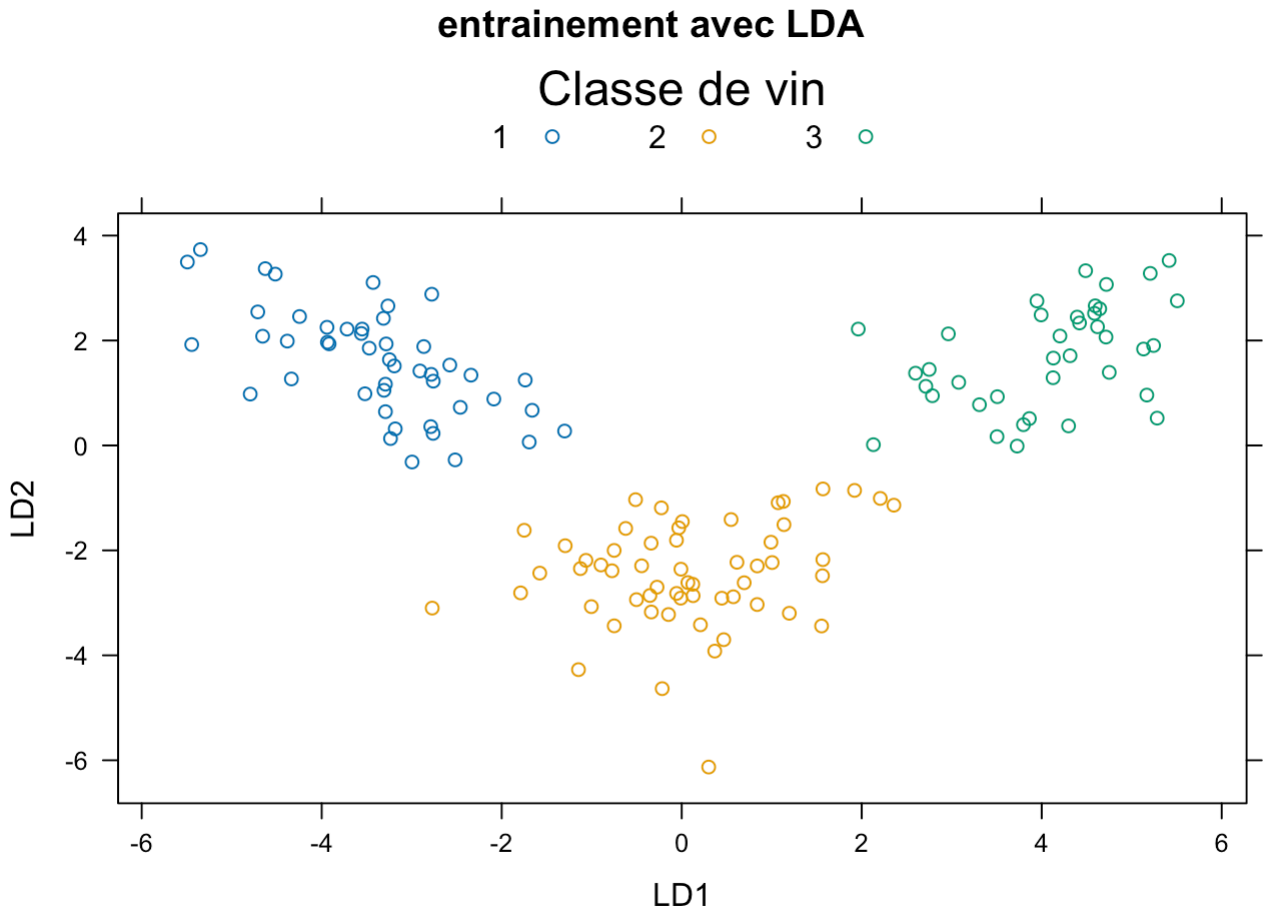
Question 7 : Le modèle SVM entraîné sur les deux composantes discriminantes de l'ALD performe bien :

1. Il classe correctement environ 97 % des vins du jeu de test, (accuracy de 0,9722).
2. la prédiction est parfaite pour les classes 1 et 3, un seul vin de la classe 2 a été mal classé.
3. Les métriques d'évaluation ont des niveaux très élevés, donc bonne discrimination des trois classes.

Donc malgré la réduction de dimension à deux variables, le modèle sépare efficacement les types de vins, ce qui confirme que l'ALD a préservé l'information discriminante essentielle.

Question 8 :

```
xyplot(LD2 ~ LD1, data = entrainement_lda2, groups = Customer_Segment, auto.key = list(title = "Classe de vin", c
olumns = 3), xlab = "LD1", ylab = "LD2", main = "entrainement avec LDA")
```

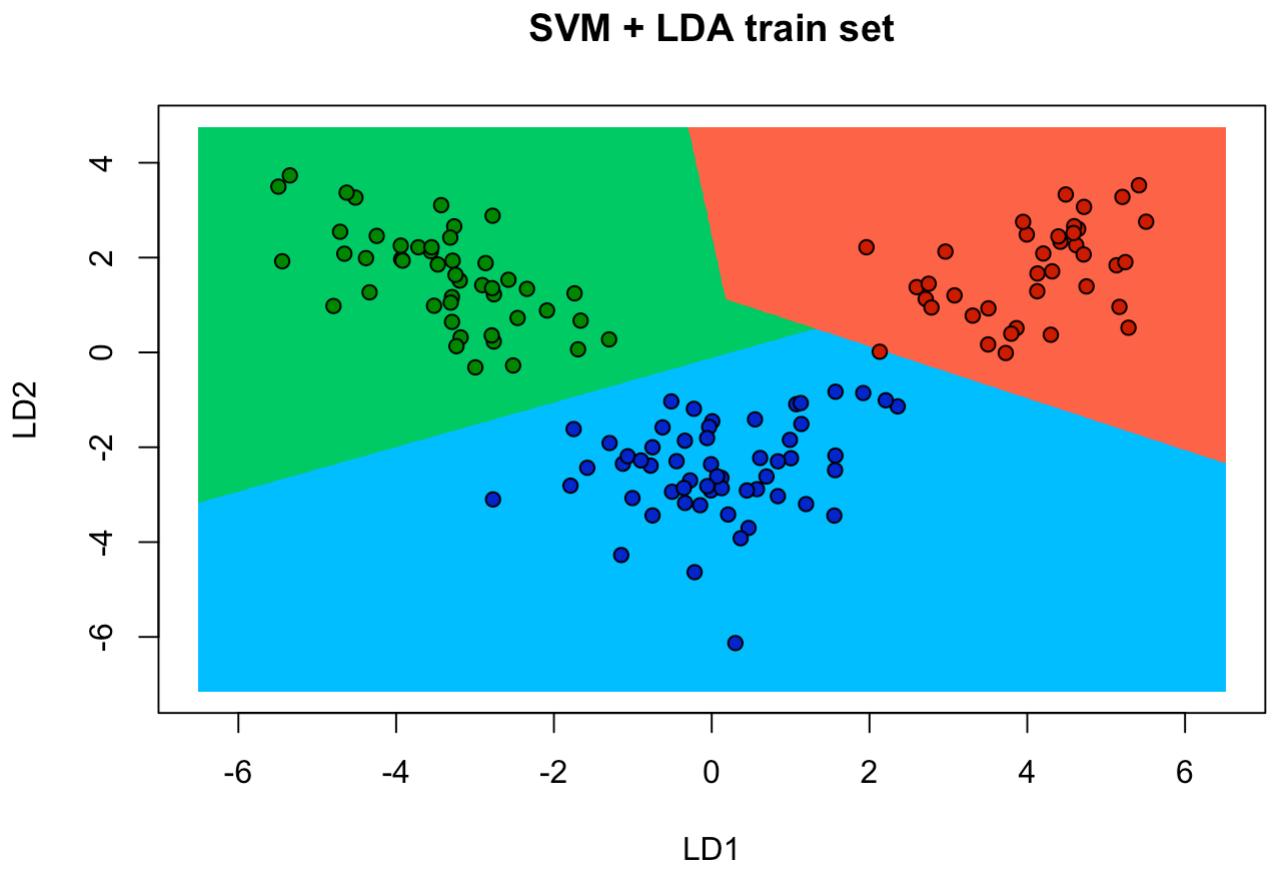


Question 9 :

```
X1 <- seq(min(entrainement_lda2$LD1) - 1, max(entrainement_lda2$LD1) + 1, by = 0.01)
X2 <- seq(min(entrainement_lda2$LD2) - 1, max(entrainement_lda2$LD2) + 1, by = 0.01)
grid_set <- expand.grid(LD1 = X1, LD2 = X2)
y_grid <- predict(svm_lda, newdata = grid_set)

plot(grid_set, pch = ".", col = ifelse(y_grid == 1, "springgreen3", ifelse(y_grid == 2, "deepskyblue", "tomat
o")), xlab = "LD1", ylab = "LD2", main = "SVM + LDA train set")

points(entrainement_lda2$LD1, entrainement_lda2$LD2,
       pch = 21,
       bg = ifelse(entrainement_lda2$Customer_Segment == 1, "green4", ifelse(entrainement_lda2$Customer_Segment =
= 2, "blue3", "red3")))
```



Question 10 :

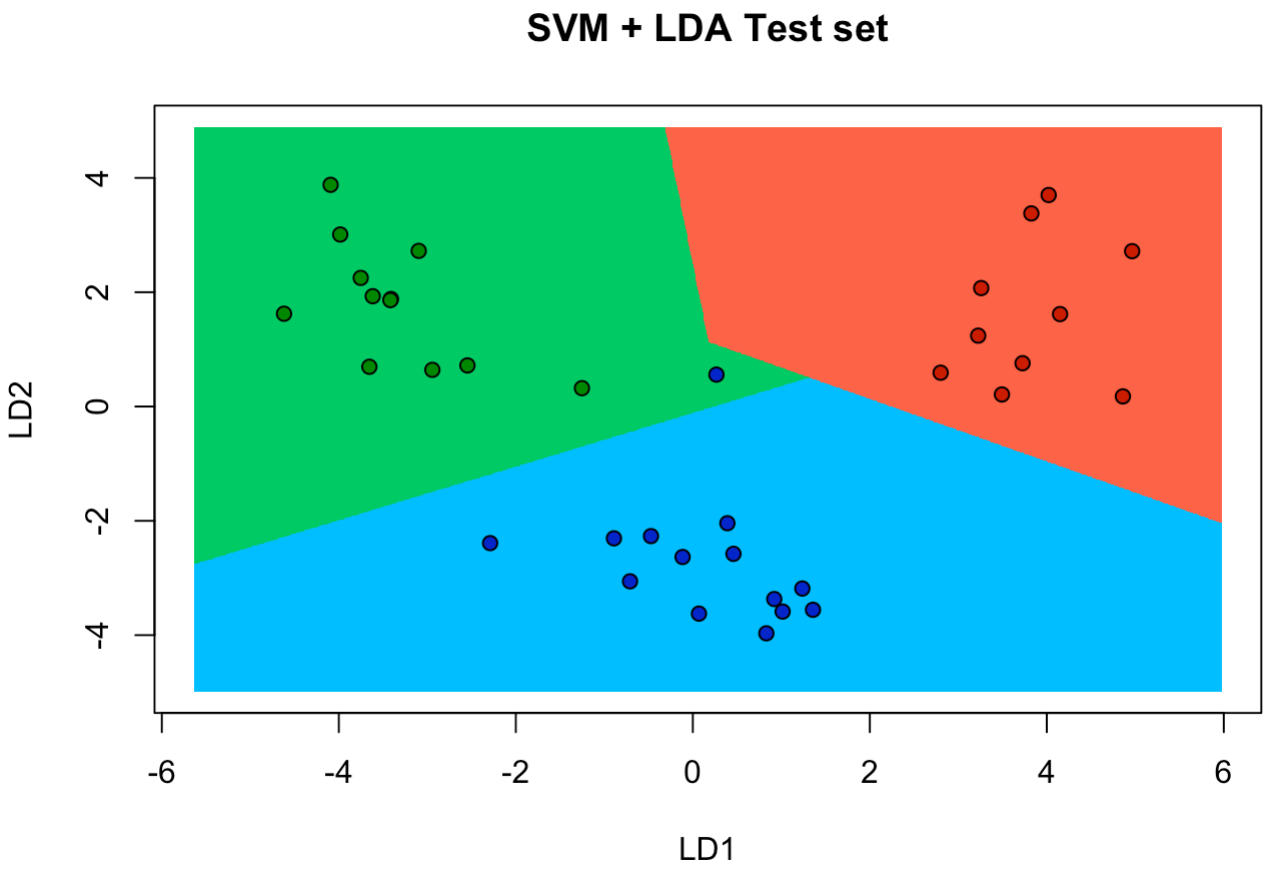
```
set_test <- test_lda2

X1 <- seq(min(test_lda2$LD1) - 1, max(test_lda2$LD1) + 1, by = 0.01)
X2 <- seq(min(test_lda2$LD2) - 1, max(test_lda2$LD2) + 1, by = 0.01)
grid_set <- expand.grid(LD1 = X1, LD2 = X2)

y_grid <- predict(svm_lda, newdata = grid_set)

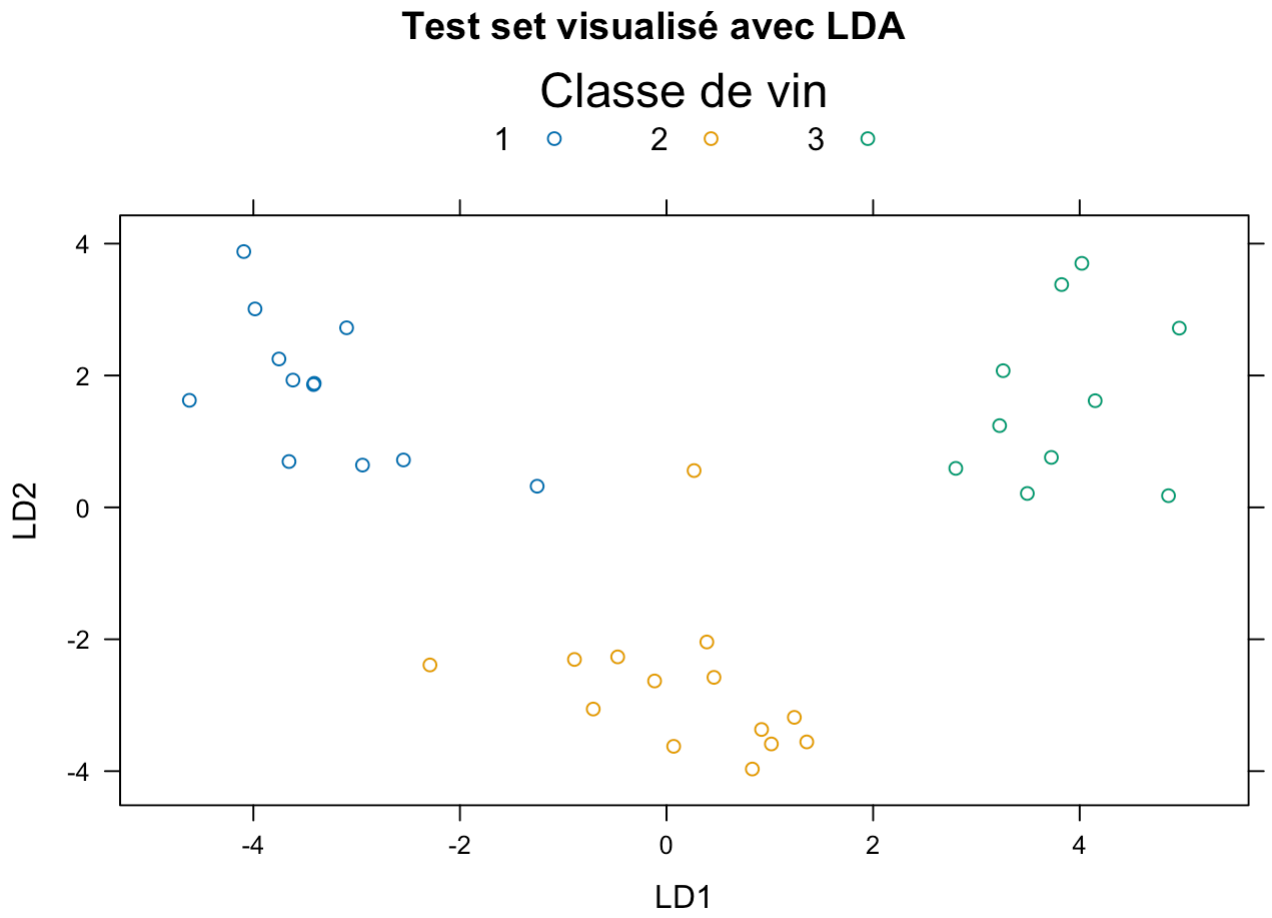
plot(grid_set, pch = ".", col = ifelse(y_grid == 1, "springgreen3", ifelse(y_grid == 2, "deepskyblue", "tomat
o")), xlab = "LD1", ylab = "LD2", main = "SVM + LDA Test set")

# Ajouter les vrais points du test set par-dessus
points(test_lda2$LD1, test_lda2$LD2, pch = 21, bg = ifelse(test_lda2$Customer_Segment == 1, "green4", ifelse(test
_lda2$Customer_Segment == 2, "blue3", "red3")))
```




```
library(lattice)

xyplot(LD2 ~ LD1, data = test_lda2, groups = Customer_Segment, auto.key = list(title = "Classe de vin", columns =
3), xlab = "LD1", ylab = "LD2", main = "Test set visualisé avec LDA")
```



Conclusion : L'objectif de ce TP était d'analyser l'impact de la réduction de dimensionnalité (par ACP et ADL) sur les performances d'un classifieur SVM appliqué à la classification des vins. Les principaux enseignements de cette étude sont les suivants :

Efficacité de la réduction de dimensionnalité : Le passage de 13 variables à seulement 2 composantes (via l'ACP ou l'ADL) a permis de simplifier drastiquement le modèle sans perte notable d'information, ce qui témoigne de la forte corrélation et redondance des caractéristiques chimiques initiales.

Performances comparées des modèles :

Données initiales (13 variables) : accuracy de 97,22 %.

ACP (2 variables, approche non supervisée) : exactitude parfaite de 100 %, démontrant sa capacité à capturer la variance essentielle pour séparer idéalement les classes.

ADL (2 variables, approche supervisée) : exactitude de 97,22 %, avec une excellente discrimination et une seule erreur d'attribution sur le jeu de test.

Apport de la visualisation : La projection en deux dimensions s'est révélée indispensable pour tracer et interpréter graphiquement les frontières de décision linéaires du SVM — une visualisation rendue impossible par la complexité de l'espace initial à 13 dimensions. Les graphiques ont permis de valider visuellement la compacité et la netteté des groupes.

Bilan final : Bien que l'ACP ait légèrement surperformé sur ce cas d'usage (100 % de précision), les deux méthodes s'avèrent extrêmement efficaces. Elles permettent d'allier simplicité structurelle, rapidité d'exécution et forte interprétabilité, tout en maintenant un niveau de performance exceptionnel (> 97 %).